

# Where the redundancy isn't

The goal was to run a language model on a tenth of the resources by exploiting redundancy. Phase 0 built the instruments and measured how much redundancy is actually there. **The answer is 1.8× — and a second model came in lower still, at 1.4×.** Here is the evidence, and the one experiment that decides whether the moonshot survives it.

Device Intel i7-1165G7 · 4c/8t · AVX-512+VNNI · 31 GB LPDDR4x · no dGPU  
Subjects TinyLlama-1.1B-Chat · Llama-3.2-1B    Corpus wikitext-2 test, 16 384 tokens  
Date 2026-09-01

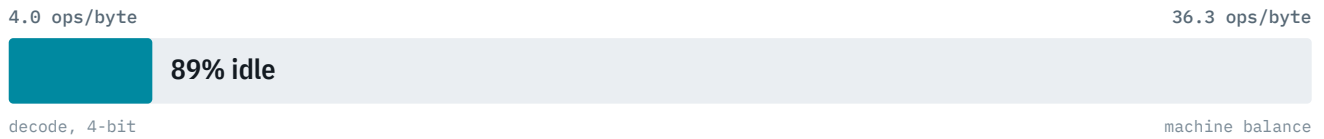
| ARITHMETIC IDLE                                            | PREFILL ÷ DECODE                                            | IN THE CACHE HIERARCHY                                          | IMPLIED CEILING                                                              |
|------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------|
| 89%                                                        | 64×                                                         | 10.7×                                                           | 1.4–1.8×                                                                     |
| of the machine's int8 throughput is unused during decoding | throughput gap on identical weights — the whole opportunity | L2/L3 vs DRAM bandwidth — a 10× that needs no redundancy at all | across both models, from every measured lever combined, at a 5% error budget |

## 01 The bottleneck is bytes, not FLOPs

Generating one token at batch size 1 reads every weight exactly once and does about two operations per weight. At 4-bit that pins arithmetic intensity at **4.0 operations per byte**, independent of model size — against a measured machine balance of 36.3. **The processor spends 89% of decoding waiting on memory.** Removing arithmetic removes idle time.

## Decode uses one ninth of the machine it runs on

operations per byte · measured int8 peak 1211 GOP/s ÷ measured read bandwidth 33.4 GB/s



The gap is not a tuning problem. It is what batch-size-1 decoding *is*: a memory scan with a little arithmetic attached.

The same fact, measured end-to-end on llama.cpp with identical weights — the only difference is whether tokens are processed in a batch (prefill) or one at a time (decode):

## Same model, same weights, 64× apart

llama2-7B Q4\_0 via ollama · tokens per second, log scale



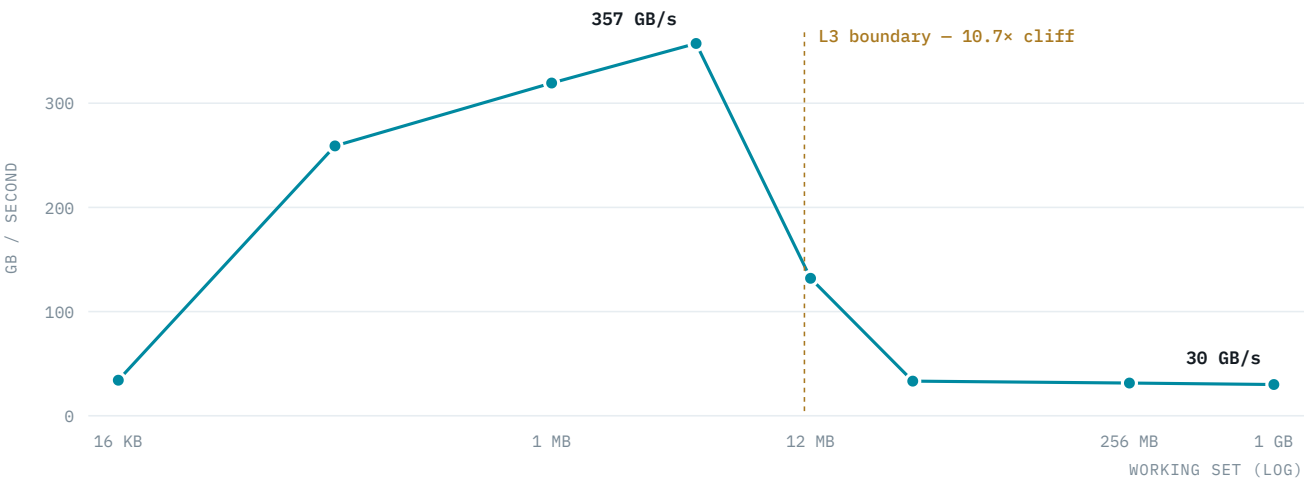
Prefill is compute-bound and fast. Decode is bandwidth-bound and slow. Every token a user waits for is on the bottom bar.

## 02 The memory hierarchy already contains a 10×

Before touching the model at all: a working set that stays resident in L2/L3 streams at **357 GB/s**. The same scan from DRAM gets **33 GB/s**. That ratio is 10.7× — the project's entire target, sitting in the cache hierarchy, requiring no redundancy whatsoever.

## Read bandwidth against working-set size

8 threads · sequential AVX-512 scan · median of 7 runs



The cliff between 4 MB and 32 MB is the L3 boundary. A decode step whose per-token working set fell to the left of it would win an order of magnitude without removing a single parameter. That is a scheduling and layout problem, not a model problem.

### ALSO MEASURED

Read bandwidth peaks at **4 threads** and drops 17% at 8 — SMT siblings contend for the same load ports and memory queue. Every tool in this repo defaults to 4.

## 03 The baseline we have to beat

Beating fp16 HuggingFace `transformers` is free and proves nothing — quantization alone does it. The honest baseline is the tuned 4-bit stack people already run.

| MEASUREMENT                      | VALUE     | NOTE                                          |
|----------------------------------|-----------|-----------------------------------------------|
| llama.cpp decode, llama2-7B Q4_0 | 5.5 tok/s | warm, median of 2 runs                        |
| Roofline ceiling for that model  | 8.8 tok/s | 3.8 GB ÷ 33.4 GB/s                            |
| Fraction of roofline achieved    | 63%       | only 1.6× of systems overhead left to reclaim |
| 10× target                       | 55 tok/s  | the number that counts                        |

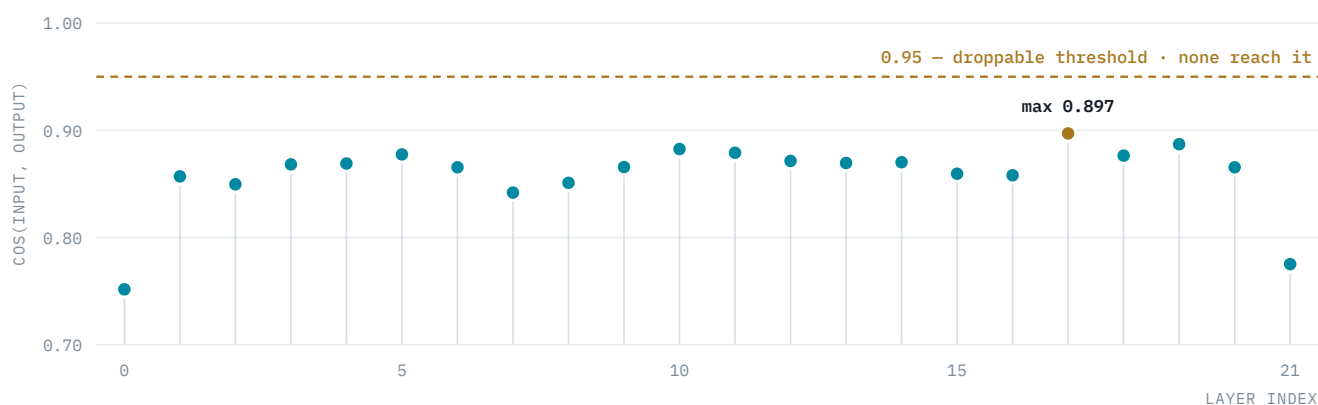
llama.cpp is already close to the memory wall. There is no slack in the implementation to harvest — **any 10× has to come from moving fewer bytes.**

## 04 How much redundancy is actually there

Five probes, each asking the same question: **which bytes did this component not need for this token?** Every number below is an *exact reconstruction error* — the sublayer output rebuilt from its top-k components and compared against the truth, expressed as a fraction of the residual-stream norm. Importance proxies like activation magnitude ignore cancellation between the terms you keep, and systematically overstate how prunable a model is.

### Depth: no layer is close to redundant

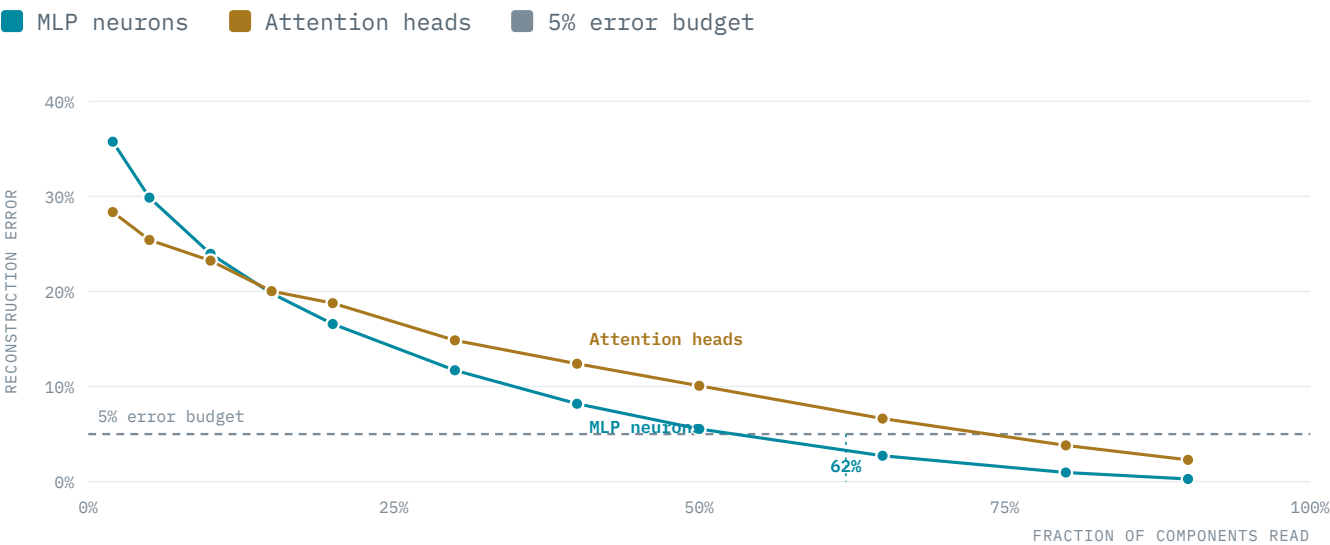
cosine similarity between each layer's input and output · 22 layers



Published layer-dropping results (ShortGPT, Gromov et al.) find layers above 0.99 in 7B–70B models. TinyLlama's most redundant layer reaches 0.897 and **not one crosses 0.95**. Every layer is doing work.

## Width: contextual sparsity is real, and small

mean exact reconstruction error vs fraction of components read, averaged over 22 layers



To stay within 5% error you must still read **62% of MLP neurons** and 65–90% of attention heads. The familiar "80–95% of neurons are inactive" result comes from ReLU-family models; SwiGLU models like this one are far denser.

| SPECULATIVE-DECODING HEADROOM | EASY FRACTION | MEAN CONSECUTIVE RUN |
|-------------------------------|---------------|----------------------|
| top-1 probability > 0.5       | 0.50          | 0.98                 |
| top-1 probability > 0.8       | 0.29          | 0.41                 |
| top-1 probability > 0.95      | 0.18          | 0.21                 |

Even an *oracle* drafter that guesses every easy token averages 1.41 tokens per verification pass — about **1.4×**, **not the 3×** reported on GPUs. This is the one lever most likely to improve on real traffic: wikitext is high-entropy prose, while chat and code are far more predictable.

### NOT LOW-RANK EITHER

The residual stream needs 546–1785 of its 2048 dimensions for 99% of energy. Middle layers do show one direction holding 95% of variance — but that is the known massive-activation outlier feature, not compressible structure.

## 05 The gate works, and rank statistics lie

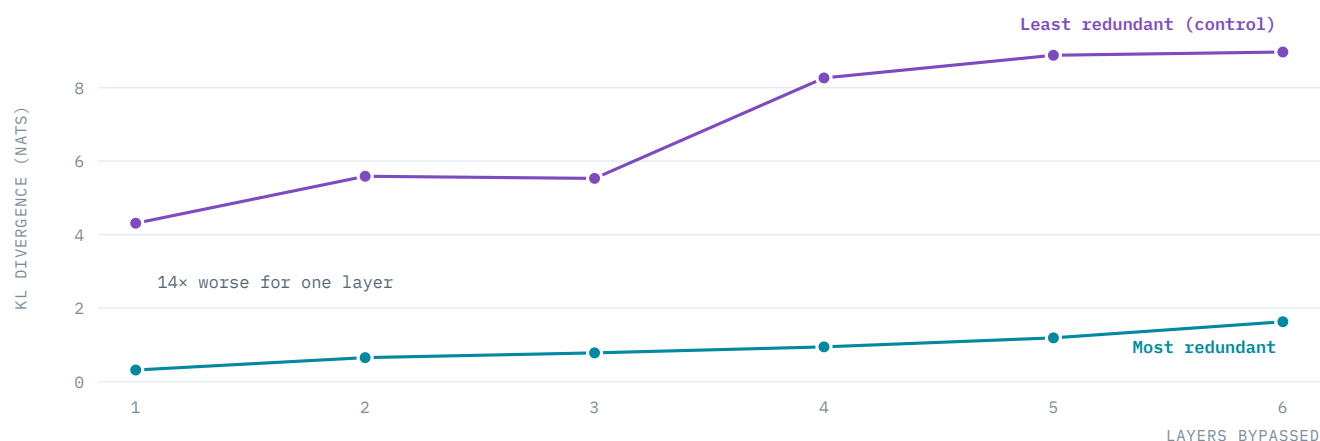
"Retain 95% intelligence" has to be an operational number before anyone has a stake in the answer. The gate scores **KL divergence from the unmodified model's full output distribution**, at every position. The self-test validates it two ways.

### The atlas predicts which layers are safe to remove

KL(reference || modified) in nats, by number of bypassed layers

■ Bypass most-redundant first

■ Control: least-redundant first

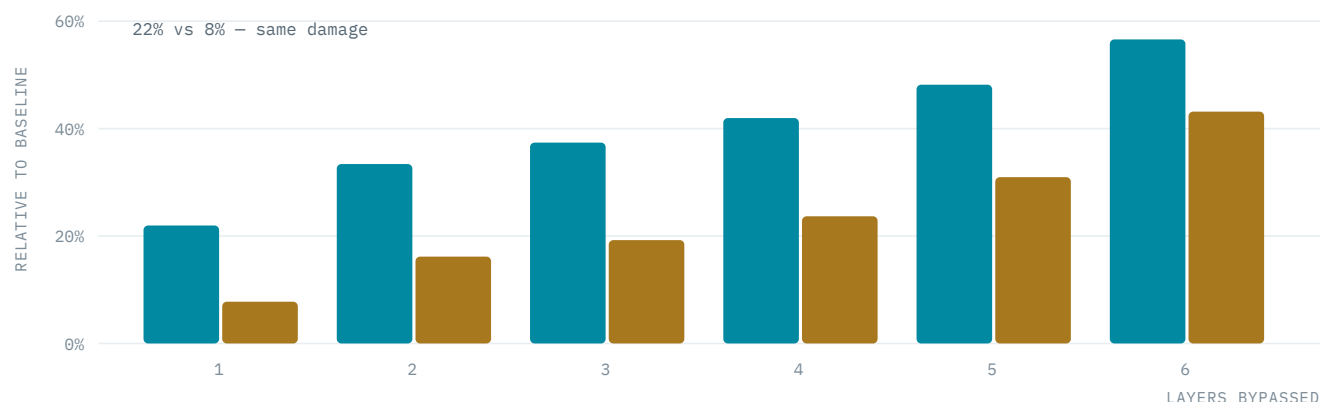


Removing the single layer the atlas rates most redundant costs KL 0.315. Removing the one it rates least redundant costs 4.31 — **14x worse**. The angular-distance metric is a validated predictor. But even the best layer to drop is not free: no layer in this model is.

## What each instrument reports about the same damage

bypassing the most-redundant layers · every metric normalized to its own baseline

■ Distribution changed (1 - top-1 agreement) ■ Ground-truth accuracy lost



At one dropped layer, **22% of tokens changed their argmax while ground-truth accuracy fell only 8%**. A multiple-choice benchmark is the same kind of rank statistic over 4 options instead of 32 000 — it would have reported almost nothing.

And the qualitative check is worse than useless. With **27% of its layers removed** and its output distribution rearranged past recognition, the model still writes fluent English:

INTACT MODEL

KL 0 · top-1 agreement 1.00

“that they are trained on a large dataset, which requires a lot of computational resources. This means that the model needs to be trained on a large da...”

6 OF 22 LAYERS BYPASSED

KL 1.63 · top-1 agreement 0.43

“that the training of the model is not optimized for the task. This means that the model is not trained to solve the problem.”

Eyeballing output would have passed this model. **Gate on KL.**

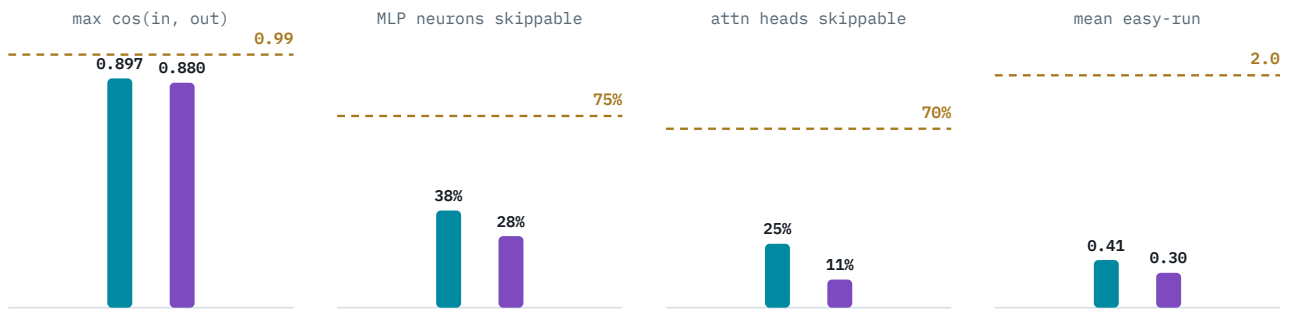
## 06 Control: was that one model's quirk?

Everything above is one model, so the obvious objection is that TinyLlama is simply unusual. Re-running the identical measurement on **Llama-3.2-1B** — same corpus, same 16 384 tokens, same settings, but a differently trained model distilled from Llama's 8B and 70B — answers it. **TinyLlama was the optimistic case.**

## Every structural signal moved the wrong way

higher is better in every panel · dashed line is what the 10× thesis needs

TinyLlama-1.1B Llama-3.2-1B What the thesis needs



Neither model has a single near-identity layer. Both need to read most of their MLP neurons and nearly all their attention heads. And the better-trained model is the *less* redundant one on all four counts.

So the 1.8× is not one model's quirk — it is a fact about models at this scale, and the ceiling for Llama-3.2-1B is **1.43×**.

### THE USEFUL PART IS A MECHANISM, NOT A NUMBER

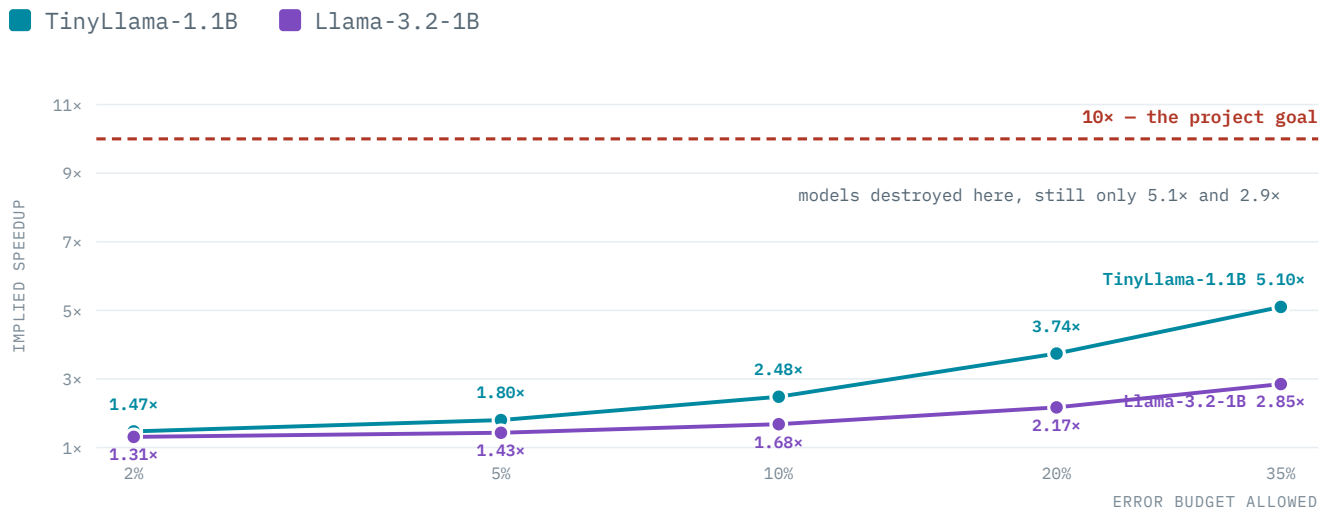
Mean cos(in, out) is **0.730** for Llama-3.2-1B against 0.859 for TinyLlama. Llama does comparable work in 16 layers instead of 22, so each layer moves the residual stream further and none comes close to an identity. That points at redundancy tracking **depth relative to task difficulty** rather than parameter count — a sharper claim than "bigger models are more redundant", and one that predicts depth redundancy reappears at 7B precisely because those models have 32 layers.

### ONE COUNTER-SIGNAL, NOT YET A LEVER

Llama-3.2-1B's residual stream is far more concentrated: 389 of 2048 dimensions carry 99% of variance, against TinyLlama's 1229. But the top direction alone holds 89.6% of it, so this is mostly the massive-activation outlier again — and it measures *activations*, while decode bytes are *weights*. Converting it into savings would need low-rank weight structure, which no probe here has tested.

## Implied speedup against the error budget you allow

naive product of every measured lever · 10× target shown for reference



Even at a 35% error budget — damage that destroys either model — the ceiling is 5.1× and 2.9×.  
**10× is not reachable on either model at any error budget, by any combination of these levers.**

### PHASE 0 DELIVERABLE

## 1.8× and 1.4× implied ceilings, against a 10× goal

And 1.8× is an upper bound twice over. The levers are assumed to compose, which no experiment here tested. And each is assumed to have a kernel that actually skips the memory traffic — none is written. The only component safe to bank is speculative decoding, at **1.24×** and **1.15×**, because it is lossless.

This does not say the moonshot is dead. It says **both experiments ran on the worst plausible subjects**. These are 1B models trained on trillions of tokens — extraordinarily over-*trained*, and therefore minimally over-*parameterized*. Redundancy is a consequence of over-parameterization, so we measured the models with the least of it available.

08

# The one experiment that decides it

Every number in this report sits at ~1B parameters, now measured twice. **Does redundancy scale with model size?** Run the same atlas on a 7B and a 13B — roughly two hours each on this laptop — and plot each signal against parameter count.

| SIGNAL                            | TINYLLAMA / LLAMA-3.2 | THESIS SURVIVES IF 7B/13B SHOWS |
|-----------------------------------|-----------------------|---------------------------------|
| max cos(layer in, out)            | 0.897 / 0.880         | 0.99+, across several layers    |
| MLP neurons needed @ 5% error     | 62% / 72%             | 20–30%                          |
| Attention heads needed @ 5% error | 75% / 89%             | ~30%                            |
| Mean easy-run, p > 0.8            | 0.41 / 0.30           | 2+                              |

As far as we can tell **nobody has published a scaling law for redundancy**, and it is a result either way. If the curves open up, ShootingStar is a systems project and the roadmap holds. If they are flat in scale, the redundancy is not there — and the work should turn to the 10.7× already sitting in the cache hierarchy, which needs no redundancy at all.

## SECOND EXPERIMENT, EQUALLY CHEAP

Rerun the atlas on real workload text rather than wikitext. Redundancy is a property of a model *on a distribution*, and measuring it on the wrong one is how published pruning results fail to reproduce. The speculative lever in particular was measured close to its worst case.

Reproduce with `scripts/phase0.sh` · ~25 min on a 4-core laptop.  
Raw records in `results/` · method and rules in `docs/ROADMAP.md`